# CS425: Computer Networks

## Homework-1

Ansh Adarsh
230157

**PROCEDURE:**

**Question 1.**(20 points)
Write a program (in any language) that generates an $n$-bit frame for transmission from a k-bit data block D and a ($n - k + 1$) bit CRC pattern P. Compile and run the program with at least two set of inputs to confirm that this program is generating CRC patterns correctly. Now, modify the program that performs the following steps:

(a) Generates a message of $k = 10$ bits.

(b) Uses the previous code with $P = 110101$ to generate the corresponding 15-bit frame $T$ for transmission.

(c) Generates transmission errors at any bit positions of $T$.

(d) Applies CRC to the received frame (i.e. frame $T$ after introducing errors) to determine if the frame should be accepted or discarded.

Attach your code with your report, and also put comments stating the instructions of how to run it.

*Solution:* The source file is `HW1.py`. It is a single Python program that uses only the Python standard library. Run it with the Python interpreter, for example:

python3 HW1.py

(If your system uses `python` for Python 3, you can use `python HW1.py`. On Windows you can also use `py HW1.py`.)

This will start the interactive program.
If you prefer to run it directly on Unix-like systems, add the shebang line `#!/usr/bin/env python3` at the top of HW1.py, make it executable with `chmod +x HW1.py`, and then run:

./HW1.py

When started the program prompts you to choose between two options: verify CRC generation (press `1`) or run the 15-bit transmission/error test (press `2`). Choosing `2` makes the program create a 10-bit message, compute its FCS using the polynomial `P`, flip bits in the 15-bit transmit frame according to a fixed error probability, and then check the received frame to decide acceptance or rejection. If you choose `1`, the program asks whether you want to enter values for `n` and `k`; if you decline, it uses the defaults `n = 10` and `k = 6`. The code is written so you can easily change defaults or the error-generation routine to handle different frame/message/polynomial sizes.

**Question 2.**(10 points)
In the Go-back-$N$ ARQ mechanism using $k$-bit sequence numbers, why is the window size limited to $2^k - 1$ and not $2^k$?

*Solution:* In Go-Back-N ARQ, if sequence numbers are represented using **k bits**, then the total sequence number space is from 0 to $2^k - 1$, i.e., there are $2^k$ distinct sequence numbers.

Now suppose the sender window size is equal to $2^k$. In that case, the sender can transmit an entire cycle of sequence numbers:
$0,1,2,\ldots,2^k - 1$.

Assume the receiver successfully receives all these frames and sends an acknowledgment. However, if this acknowledgment is lost, the sender will eventually time out and retransmit the same frames again (starting from 0).

The problem arises because sequence numbers are taken **modulo $2^k$**. After receiving the full window, the receiver's expected sequence number wraps around to 0 again. Therefore, when the sender retransmits frames starting from 0, the receiver cannot distinguish whether these frames belong to a *new transmission cycle* or are simply duplicates of the previous frames. As a result, duplicate frames may be accepted, causing an error.

Now consider the case when the window size is limited to $2^k - 1$. The sender transmits frames
$0,1,2,\ldots,2^k - 2$.
After receiving them, the receiver expects frame number $2^k - 1$.

If the acknowledgement is lost and the sender retransmits starting again from 0, the receiver is expecting $2^k - 1$, not 0. Therefore, when it receives frame 0 again, it correctly identifies it as an old or duplicate frame and discards it. No ambiguity occurs.

Thus, to prevent confusion between new frames and retransmitted old frames, the sender window size in Go-Back-N must be limited to $2^k - 1$ and cannot be equal to $2^k$.

**Question 3.**(10 points)
What is the maximum window size that can be used in the Selective-Reject ARQ mechanism that uses k-bit sequence numbers? Explain your answer.

*Solution:* In Selective-Reject (Selective-Repeat) ARQ, if sequence numbers are represented using **k bits**, then the total number of distinct sequence numbers available is
$0,1,2,\ldots,2^k - 1$.
So, the size of the sequence number space is $2^k$.

Let the sender window size be w. In Selective-Repeat, the receiver accepts frames out of order and buffers them. Therefore, it is very important that a frame from a previous transmission cycle is never mistaken for a new frame after sequence numbers wrap around (modulo $2^k$).

Now consider two consecutive sender windows. The first window contains www frames, and after acknowledgment, the next window also contains w new frames. Together, these two windows involve 2w distinct sequence numbers (if there is no overlap).

If 2w > $2^k$, then because numbering is done modulo $2^k$, some sequence numbers in the second window will overlap with those of the first window. In such a situation, if acknowledgments are lost and the sender retransmits old frames, the receiver may confuse these old frames with new ones (since their sequence numbers match). This creates ambiguity and can lead to incorrect acceptance of duplicate frames.

To avoid this problem, the two consecutive windows must not overlap in sequence numbers. Therefore, the total distinct numbers used by both windows must not exceed the sequence space:

2w ≤ $2^k$.

Solving this inequality:

w ≤ $2^{k-1}$.
Hence, the **maximum window size in Selective-Reject ARQ with k-bit sequence numbers is** $2^{k-1}$

This ensures that even after wraparound, old frames cannot be confused with new ones.

**Question 4.**(10 points)
A channel has a data rate of 4 kbps and a propagation delay of 20 ms. For what range of frame sizes does stop-and-wait give an efficiency of at least 50%?

*Solution:* We Given:

- Data rate R=4 kbps =4000 bits/s
- One-way propagation delay $t_{prop}$=20 ms =0.02 s
- Required efficiency U≥50% (i.e. U≥0.5)

Background formula (stop-and-wait):

$U = 1/(1+2a)$, where $a = t_{prop}/t_{frame}$ and $t_{prop}$ is the frame transmission time.

Step 1 — Apply the efficiency requirement

$0.5 \le 1/(1+2a)$.

Take reciprocals

$1+2a \le 2 \Rightarrow 20\ a \le 1/2$ .

Step 2 — Replace $a = t_{prop}/t_{frame}$

$t_{\text{prop}}/t_{\text{frame}} \leq 1/2$

Substitute $t_{\text{prop}}$ = 20 ms:

$20 \text{ ms.}/t_{\text{frame}} \leq 1/2 \Rightarrow 40 \text{ ms.} \leq t_{\text{frame}}.$

Step 3 — Convert transmission time to frame size (bits)
Transmission time and frame size S are related by

With R=4000 b/s and $40 \text{ ms.} \leq t_{\text{frame}}.$

$t_{\text{frame}} = S/\text{R} \Rightarrow 40 \cdot 10^{-3} \leq S_F/(4000 \text{ bps}) \Rightarrow 160 \text{ bits} \leq S_F$

**Final answer**

Therefore, to ensure that the stop-and-wait protocol achieves an efficiency of at least 50%, the frame length must be **160 bits or greater**.

**Question 5.**(20 points)
Consider a frame consists of one character of 4 bits. Assume that the probability of bit error is $10^{-3}$ and that it is independent in each bit.

(a) What is the probability that the received frame contains no errors?

(b) What is the probability that the received frame contains at least one error?

(c) Now assume that one parity bit is added. What is the probability that the frame is received with errors that are not detected?

*Solution:* (a) The probability of bit error is given as $10^{-3}$. Let this be denoted by p. Therefore,

- Probability that a bit is in error = p = 0.001
- Probability that a bit is correct = 1 − p = 0.999

Let Ei denote the event that the ith bit is flipped (i.e., contains an error). Then,

- P(Ei) = p
- P(Ei complement) = 1 − p

We are interested in finding the probability that none of the four bits are in error. In other words, we want the probability that all four bits are received correctly:

P(E1 complement ∩ E2 complement ∩ E3 complement ∩ E4 complement)

Since the bit errors are independent, we multiply the individual probabilities:

P(no error in all 4 bits) = (1 − p)^4
= 0.999^4

Using numerical evaluation,

0.999^4 ≈ 0.996

Thus, the probability that the 4-bit frame is received without any error is approximately 0.996.

(b) The event “at least one bit is in error” is the complement of the event “no bit is in error,” which was calculated in part (a).

Let A denote the event that the received frame contains at least one error.

From part (a), we already found:

P(no error) ≈ 0.996

Since these two events are complementary,

P(A) = 1 − P(no error)

Substituting the value,

P(A) = 1 − 0.996
P(A) ≈ 0.004

Thus, the probability that the 4-bit frame contains at least one error is approximately

0.004, which is equal to 4 × $10^{-3}$.

Now one parity bit is added, so total bits per frame = 5.

(c) A single parity bit detects all **odd** numbers of bit errors (1, 3, or 5 errors).
However, it **fails to detect** errors when the total number of flipped bits is even (2 or 4 errors).

So we calculate:

P(undetected error) = P(2 errors) + P(4 errors)

Using the binomial formula:

$$P(\bigcap_{k\notin\{i,j\}} \overline{E_k}) = \prod_{k\notin\{i,j\}} P(\overline{E_k}) = (1-p)^3$$

Finally we will got,

$$\prod_{i \neq j} P(E_i \cap E_j) \cdot P(\bigcap_{k \notin \{i,j\}} E_k)$$
$$= \prod_{i \neq j} p^2 \cdot (1-p)^3$$
$$= \binom{5}{2} \cdot p^2 \cdot (1-p)^3$$
$$\approx 10 \cdot 10^{-6} \cdot 0.997$$
$$P(\text{2 errors}) = 9.97 \cdot 10^{-6}$$

Probability of exactly 4 errors:

$$\binom{5}{4} \cdot p^4 \cdot (1-p)^1$$
$$= 5 \cdot p^4 \cdot (1-p)$$
$$= 4.995 \cdot 10^{-12}$$

Finally, we get the probability of error not being detected as,

$P(2 \text{ errors}) + P(4 \text{ errors}) = 9.97 \cdot 10^{-6} + 4.995 \cdot 10^{-12} \approx 9.97 \cdot 10^{-6}$

**Question 6.**(10 points)
For P = 110011 and M = 11100011, find the CRC.

*Solution:* Given

- Generator polynomial P = 110011
- Message M = 11100011
- Since the length of P is 6 bits, we append 5 zeros (|P| − 1) to the message before division.

Step 1: Padding the Message
Append five zeros to M: 11100011 00000

This becomes the dividend for the CRC calculation.

Step 2: Perform Modulo-2 Division
Divide the padded message by the divisor P = 110011 using binary (modulo-2) long division.

- Subtraction is done using XOR (no borrow).
- At each step, align the divisor with the leftmost 1 of the current working bits.
- Continue the division until all bits are processed.

$$
\begin{array}{r}
10110110 \\
P = 110011 \overline{)\ 11100011\,00000} \\
\underline{110011} \\
010111 \\
\underline{000000} \\
101111 \\
\underline{110011} \\
111000 \\
\underline{110011} \\
010110 \\
\underline{000000} \\
101100 \\
\underline{110011} \\
111110 \\
\underline{110011} \\
011010 \\
\underline{000000} \\
R = 11010
\end{array}
$$

After completing the division:

- Quotient obtained = 10110110
- Remainder obtained = 11010

Step 3: Determine the CRC
The CRC is defined as the remainder obtained from the modulo-2 division.

Therefore, CRC = 11010

If required, the final transmitted codeword is formed by appending the CRC to the original message: 11100011 11010

**Question 7.**(20 points)

(a) In a CRC error-detecting scheme, choose $P(x) = x^4 + x + 1$. Encode the bits 10010011011.

(b) Suppose the channel introduces an error pattern 100010000000000 (i.e., a flip from 1 to 0 or from 0 to 1 in position 1 and 5). What is received? Can the error be detected?

(c) Repeat part (b) with error pattern 100110000000000.

*Solution:* Given generator polynomial: P(x) = $x^4$ + x + 1 binary form of divisor: 10011

message bits: M = 10010011011

(a) encoding the message

step 1: append zeros

since degree of P(x) = 4, append 4 zeros to the message: 10010011011 0000 this becomes the dividend.

step 2: modulo-2 long division

divisor = 10011 we perform binary division using XOR (no borrow, no carry).

initial 5 bits: 10010 since leading bit is 1, XOR with 10011:

**10010**

**10011**

00001

bring down next bit → 00001 0 → 000010 ignore leading zeros → 00010

leading bit is 0, so just shift and bring next bit → 00100 leading bit is 0, shift again → 01001 leading bit is 0, shift again → 10011 now leading bit is 1, XOR with divisor:

**10011**

**10011**

00000

bring next bit → 00000 0 → 000000
continue bringing remaining bits and repeating same process. after completing the entire division, the final remainder is: 1100

this corresponds to polynomial: R(x) = $x^3 + x^2$

step 3: form transmitted codeword

append remainder to original message: MR = 10010011011 1100 this is the transmitted frame.

$$P(x) = x^4 + x + 1 \;\overline{)\; x^{14} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 \;}\; = x^4 \cdot M(x)$$

$$\begin{array}{l} x^{10} + x^6 + x^4 + x^2 \\ x^{14} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 \\ \underline{x^{14} + x^{11} + x^{10}} \\ \quad x^{10} + x^8 + x^7 \\ \quad \underline{x^{10} + x^7 + x^6} \\ \quad\quad x^8 + x^6 + x^5 + x^4 \\ \quad\quad \underline{x^8 + x^5 + x^4} \\ \quad\quad\quad x^6 \\ \quad\quad\quad \underline{x^6 + x^3 + x^2} \\ \quad\quad\quad R(x) = x^3 + x^2 \end{array}$$

(b) error pattern = 100010000000000 this means bit 1 and bit 5 are flipped.

step 1: find received word - W = MR XOR error pattern

**MR = 100100110111100**

**error = 100010000000000**

W = 000110110111100

step 2: divide received word by 10011

perform modulo-2 division again. after completing division, remainder = 1110 since remainder ≠ 0000, error is detected.

why?
because a valid transmitted codeword must be exactly divisible by the generator polynomial.
a non-zero remainder means corruption occurred.

$$
\begin{array}{r}
 & 11001110 \\
P = 10011 \overline{)\ 000110110111100} \\
 & \underline{10011} \\
 & 10000 \\
 & \underline{10011} \\
 & 11111 \\
 & \underline{10011} \\
 & 11001 \\
 & \underline{10011} \\
 & 10100 \\
 & \underline{10011} \\
 & R = 1110
\end{array}
$$

(c) error pattern = 100110000000000

step 1: received word

**MR = 100100110111100**

**error = 100110000000000**

W = 000010110111100

step 2: divide by 10011

$$
\begin{array}{r}
 & 1010100 \\
P = 10011 \overline{)\ 000010110111100} \\
 & \\
 & \underline{10011}
\end{array}
$$

after performing modulo-2 division, remainder = 0000 since remainder = 0,
error is not detected.
CRC detects an error if the error polynomial is not divisible by the generator polynomial.

in part (b), the error polynomial is not divisible by P(x), so detection happens.

in part (c), the error polynomial is divisible by P(x), so the receiver cannot detect it.